

ИСТОК — Архитектура мультиагентной системы

Executive Summary

Исток — кастомная мультиагентная система на Go (Clean Architecture), спроектированная для универсальных задач: генерация кода, контент, аналитика, визуальные ассеты, DevOps. Система из **10 специализированных агентов**, организованных в 4 уровня (Control Plane → Planning → Execution → Verification), общается через типизированные JSON-сообщения по Event Bus. Жизненный цикл каждой задачи управляется конечным автоматом (FSM) с retry logic, circuit breaker и экспоненциальным backoff.

Ключевые решения

- **10 агентов** в 4-уровневой иерархии — минимум связности, максимум специализации
 - **Event Bus** на Go channels (+ Redis Streams для persistence) — zero external dependencies в core
 - **FSM с 11 состояниями** — полный контроль над жизненным циклом задачи
 - **Clean Architecture** — бизнес-логика изолирована от инфраструктуры, LLM-провайдеры подключаются через интерфейсы
 - **Exponential backoff + circuit breaker** — отказоустойчивость без каскадных сбоев
-

1. Роли агентов и иерархия



Layer 0 — Control Plane

Агент	Роль	Ответственность
-------	------	-----------------

Orchestrator	Центральный диспетчер	Маршрутизация задач, управление FSM, retry logic, приоритизация очереди, агрегация результатов
--------------	-----------------------	--

Orchestrator — единственный агент, который видит полную картину. Он не генерирует контент и не пишет код. Его задача — **маршрутизация и контроль**.

Layer 1 — Planning & Analysis

Агент	Роль	Ответственность
Planner	Декомпозитор	Разбивает высокоуровневую задачу на атомарные подзадачи, определяет зависимости и порядок исполнения
Analyst	Аналитик данных	Сбор метрик, BI-запросы, построение отчётов, анализ данных
Architect	Системный архитектор	Проектирование структуры, ADR (Architecture Decision Records), выбор паттернов

Layer 2 — Execution

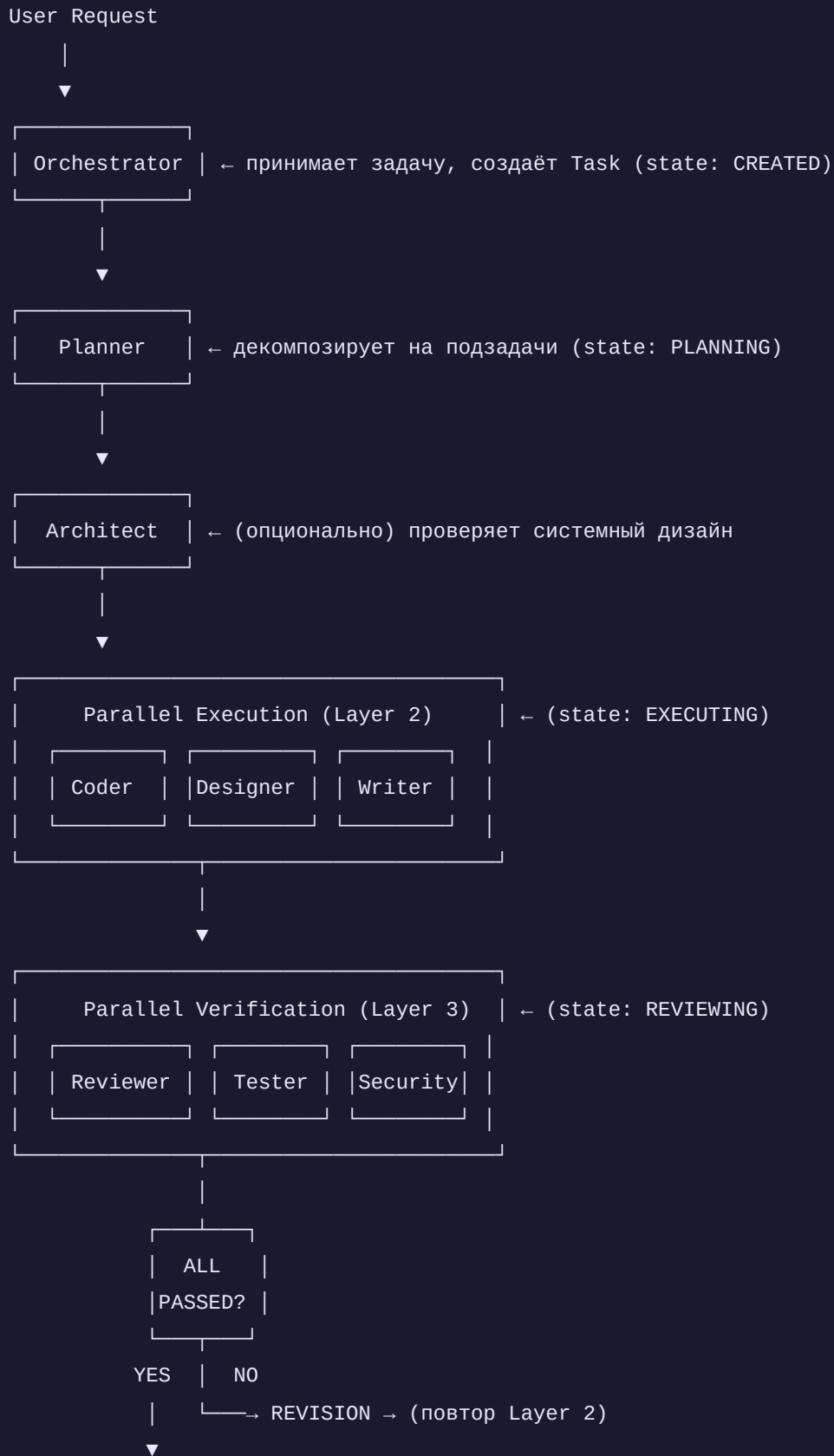
Агент	Роль	Ответственность
Coder	Генератор кода	Пишет Go, TypeScript, SQL, конфиги. Работает с AST, форматирует, запускает <code>go vet</code>
Designer	Визуальный дизайнер	Генерация UI-макетов, иконок, иллюстраций, CSS, SVG-ассетов
Writer	Контент-мейкер	Документация, копирайтинг, README, changelog, API docs
DevOps	Инфраструктура	Dockerfile, CI/CD pipelines, Terraform, Helm charts, мониторинг

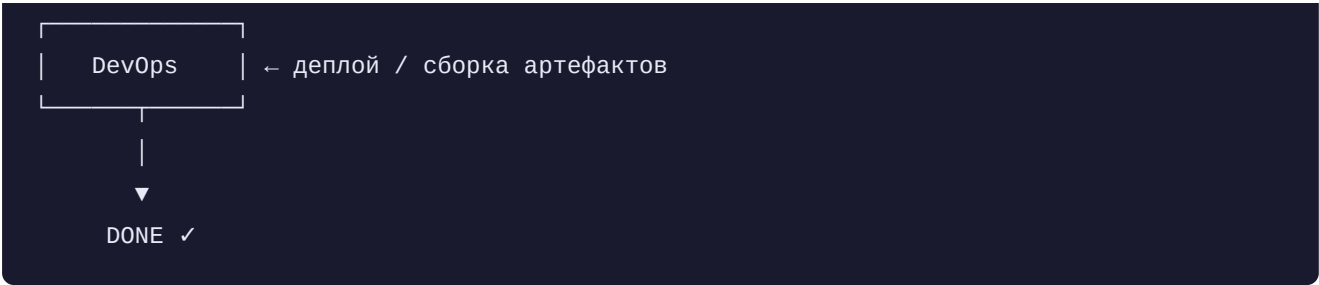
Layer 3 — Verification

Агент	Роль	Ответственность
Reviewer	Code Review	Проверка стиля, lint, архитектурных нарушений, best practices
Tester	QA	Генерация и запуск unit/integration тестов, покрытие
Security	Безопасность	SAST-анализ, поиск секретов в коде, проверка зависимостей (CVE), OWASP

2. Workflow — полный цикл задачи

2.1. Последовательность шагов



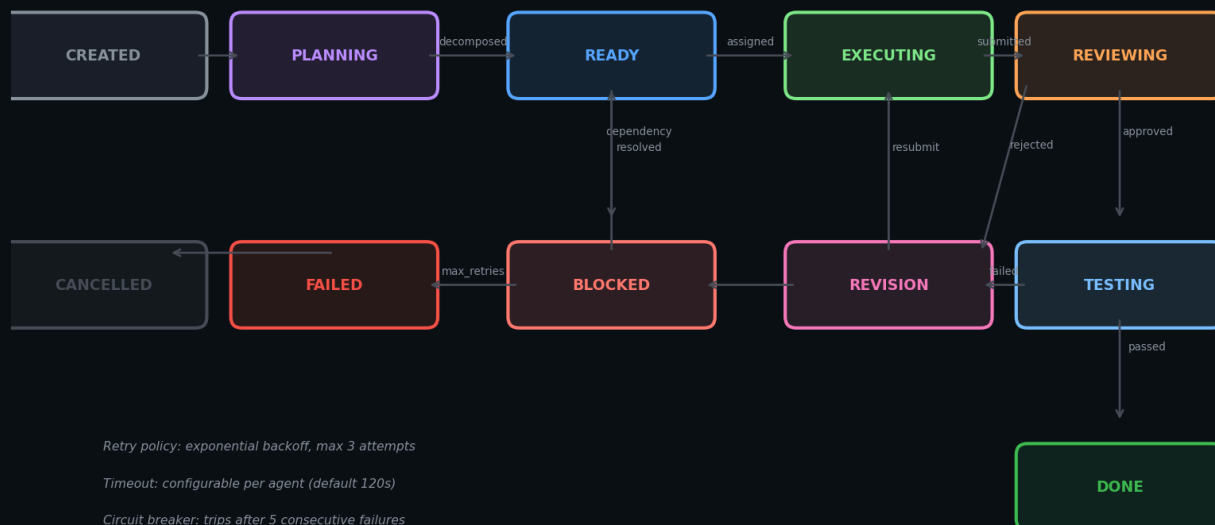


2.2. Кто ставит задачу, кто проверяет, кто генерирует

Действие	Кто инициирует	Кто исполняет	Кто верифицирует
Декомпозиция задачи	Orchestrator	Planner	Architect (опц.)
Написание кода	Planner → Orchestrator	Coder	Reviewer + Tester + Security
Генерация визуала	Planner → Orchestrator	Designer	Reviewer (визуальная часть)
Написание документации	Planner → Orchestrator	Writer	Reviewer
Деплой	Orchestrator	DevOps	Security
Аналитический отчёт	Orchestrator	Analyst	Reviewer

3. State Machine — жизненный цикл задачи

ИСТОК — State Machine: жизненный цикл задачи



3.1. Определение состояний

```
type TaskState string

const (
    StateCreated    TaskState = "CREATED"    // Задача создана, ждёт маршрутизации
    StatePlanning   TaskState = "PLANNING"    // Planner декомпозирует
    StateReady      TaskState = "READY"       // Подзадачи готовы, ждут назначения
    StateExecuting  TaskState = "EXECUTING"    // Агент-исполнитель работает
    StateReviewing  TaskState = "REVIEWING"   // Верификация результата
    StateRevision   TaskState = "REVISION"    // Отправлено на доработку
    StateTesting    TaskState = "TESTING"     // Прохождение тестов
    StateBlocked    TaskState = "BLOCKED"     // Заблокировано зависимостью/таймаутом
    StateDone       TaskState = "DONE"        // Завершено успешно
    StateFailed     TaskState = "FAILED"      // Провалено после max retries
    StateCancelled  TaskState = "CANCELLED"   // Отменено пользователем/системой
)
```

3.2. Таблица переходов

Из	В	Триггер	Условие
CREATED	PLANNING	task.accepted	Orchestrator принял задачу
PLANNING	READY	task.decomposed	Planner завершил декомпозицию
READY	EXECUTING	task.assigned	Агент-исполнитель свободен
READY	BLOCKED	task.blocked	Есть незавершённая зависимость
BLOCKED	READY	dependency.resolved	Зависимость завершена
BLOCKED	FAILED	max_retries_exceeded	Превышено кол-во попыток
EXECUTING	REVIEWING	task.submitted	Агент отправил результат
REVIEWING	TESTING	review.approved	Ревьюер одобрил
REVIEWING	REVISION	review.rejected	Найдены проблемы
REVISION	EXECUTING	task.resubmitted	Агент исправил замечания
TESTING	DONE	tests.passed	Все тесты прошли
TESTING	REVISION	tests.failed	Тесты не прошли
FAILED	CANCELLED	task.cancelled	Ручная или автоматическая отмена

3.3. Retry Policy

```
type RetryPolicy struct {
    MaxAttempts      int           `json:"max_attempts"`      // default: 3
    BaseDelay        time.Duration `json:"base_delay"`        // default: 2s
    MaxDelay         time.Duration `json:"max_delay"`         // default: 120s
    BackoffFactor    float64      `json:"backoff_factor"`    // default: 2.0
    RetryableErrors  []string     `json:"retryable_errors"`  // e.g. ["timeout", "rate_li
}
```

// Exponential backoff: delay = min(base * factor^attempt, max)
// Attempt 1: 2s, Attempt 2: 4s, Attempt 3: 8s

3.4. Circuit Breaker

```
type CircuitBreaker struct {  
    State          CBState      `json:"state"`           // CLOSED | OPEN | HALF_OP  
    FailureThreshold int         `json:"failure_threshold"` // default: 5  
    ResetTimeout    time.Duration `json:"reset_timeout"`    // default: 60s  
    FailureCount     int         `json:"failure_count"`  
    LastFailure      time.Time   `json:"last_failure"`  
}
```

Когда агент падает 5 раз подряд — circuit breaker переходит в `OPEN`, все задачи для этого агента перенаправляются на fallback или ставятся в `BLOCKED`.

4. JSON-контракты межагентного взаимодействия

4.1. Базовая структура: AgentMessage

Это **единый конверт** для всех сообщений между агентами:

```
{
  "$schema": "istok/agent-message/v1",
  "id": "msg-a7f3b2c1-4e5d-6f7a-8b9c-0d1e2f3a4b5c",
  "type": "task.assigned",
  "version": "1.0.0",
  "timestamp": "2026-04-24T14:30:00.000Z",
  "trace_id": "trace-x9y8z7w6",
  "correlation_id": "task-001",

  "source": {
    "agent_id": "orchestrator",
    "agent_type": "control_plane",
    "instance_id": "orch-01"
  },

  "target": {
    "agent_id": "coder",
    "agent_type": "executor",
    "instance_id": "coder-03"
  },

  "payload": {
    "task": {
      "id": "task-001",
      "parent_id": null,
      "title": "Реализовать REST endpoint /api/v1/users",
      "description": "CRUD операции для сущности User с валидацией",
      "priority": "high",
      "state": "EXECUTING",
      "constraints": {
        "language": "go",
        "framework": "chi",
        "style_guide": "uber-go",
        "max_file_size_lines": 300,
        "test_coverage_min": 80
      },
    },
    "context": {
      "project": "istok-api",
      "branch": "feature/users-crud",
      "related_files": [
        "internal/domain/user.go",
        "internal/repository/user_repo.go"
      ],
      "dependencies": ["task-000"]
    }
  }
}
```

```
    },  
    "deadline": "2026-04-24T15:30:00.000Z"  
  }  
},  
  
"metadata": {  
  "retry_count": 0,  
  "max_retries": 3,  
  "timeout_ms": 120000,  
  "idempotency_key": "idem-task001-coder-v1"  
}  
}
```

4.2. Planner → Orchestrator: результат декомпозиции

```
{
  "type": "task.decomposed",
  "source": { "agent_id": "planner" },
  "target": { "agent_id": "orchestrator" },
  "payload": {
    "parent_task_id": "task-001",
    "subtasks": [
      {
        "id": "task-001-a",
        "title": "Создать domain entity User",
        "assigned_to": "coder",
        "estimated_duration_ms": 30000,
        "dependencies": [],
        "priority": "high",
        "tags": ["domain", "entity"]
      },
      {
        "id": "task-001-b",
        "title": "Создать repository interface + PostgreSQL impl",
        "assigned_to": "coder",
        "estimated_duration_ms": 60000,
        "dependencies": ["task-001-a"],
        "priority": "high",
        "tags": ["repository", "postgres"]
      },
      {
        "id": "task-001-c",
        "title": "Сгенерировать OpenAPI spec",
        "assigned_to": "writer",
        "estimated_duration_ms": 20000,
        "dependencies": ["task-001-a"],
        "priority": "medium",
        "tags": ["docs", "openapi"]
      },
      {
        "id": "task-001-d",
        "title": "Создать UI-макет страницы Users",
        "assigned_to": "designer",
        "estimated_duration_ms": 45000,
        "dependencies": [],
        "priority": "medium",
        "tags": ["ui", "mockup"]
      }
    ]
  }
}
```

```
    }  
  ],  
  "execution_graph": {  
    "parallel_groups": [  
      ["task-001-a", "task-001-d"],  
      ["task-001-b", "task-001-c"]  
    ],  
    "critical_path": ["task-001-a", "task-001-b"]  
  }  
}  
}
```

4.3. Coder → Orchestrator: результат выполнения

```
{
  "type": "task.completed",
  "source": { "agent_id": "coder", "instance_id": "coder-03" },
  "target": { "agent_id": "orchestrator" },
  "payload": {
    "task_id": "task-001-a",
    "status": "success",
    "duration_ms": 28500,
    "artifacts": [
      {
        "id": "art-001",
        "type": "source_code",
        "path": "internal/domain/user.go",
        "language": "go",
        "content_hash": "sha256:a1b2c3d4e5f6...",
        "size_bytes": 2048,
        "lines_of_code": 87
      },
      {
        "id": "art-002",
        "type": "test_code",
        "path": "internal/domain/user_test.go",
        "language": "go",
        "content_hash": "sha256:f6e5d4c3b2a1...",
        "size_bytes": 3072,
        "lines_of_code": 124
      }
    ],
    "metrics": {
      "tokens_used": 4200,
      "model": "gpt-4o",
      "cost_usd": 0.042,
      "test_coverage_percent": 92.3
    },
    "self_review": {
      "confidence": 0.88,
      "known_issues": [],
      "notes": "Использовал value objects для Email и Phone"
    }
  }
}
```

4.4. Designer → Orchestrator: визуальные ассеты

```
{
  "type": "task.completed",
  "source": { "agent_id": "designer" },
  "target": { "agent_id": "orchestrator" },
  "payload": {
    "task_id": "task-001-d",
    "status": "success",
    "duration_ms": 42000,
    "artifacts": [
      {
        "id": "art-010",
        "type": "ui_mockup",
        "path": "assets/mockups/users-page.png",
        "format": "png",
        "dimensions": { "width": 1440, "height": 900 },
        "content_hash": "sha256:deadbeef...",
        "size_bytes": 245760
      },
      {
        "id": "art-011",
        "type": "design_tokens",
        "path": "assets/tokens/users.json",
        "format": "json",
        "content_hash": "sha256:cafebabe..."
      },
      {
        "id": "art-012",
        "type": "svg_icon",
        "path": "assets/icons/user-avatar.svg",
        "format": "svg",
        "content_hash": "sha256:1234abcd..."
      }
    ],
    "metrics": {
      "tokens_used": 1800,
      "model": "dall-e-3",
      "cost_usd": 0.08
    },
    "design_spec": {
      "color_palette": ["#0d1117", "#58a6ff", "#7ee787", "#e6edf3"],
      "typography": { "heading": "Inter Bold", "body": "Inter Regular" },
      "spacing_unit": 8,
    }
  }
}
```

```
    "accessibility_score": "AA"  
  }  
}  
}
```

4.5. Reviewer → Orchestrator: результат ревью

```
{
  "type": "review.completed",
  "source": { "agent_id": "reviewer" },
  "target": { "agent_id": "orchestrator" },
  "payload": {
    "task_id": "task-001-a",
    "verdict": "approved_with_comments",
    "severity_summary": {
      "critical": 0,
      "warning": 1,
      "info": 3
    },
  },
  "comments": [
    {
      "id": "cmt-001",
      "file": "internal/domain/user.go",
      "line": 42,
      "severity": "warning",
      "category": "error_handling",
      "message": "Validate() возвращает первую ошибку. Лучше аккумулировать все ошибки",
      "suggestion": "Использовать hashicorp/go-multierror или кастомный ValidationError",
      "auto_fixable": true
    },
    {
      "id": "cmt-002",
      "file": "internal/domain/user.go",
      "line": 15,
      "severity": "info",
      "category": "naming",
      "message": "Поле 'Fname' лучше назвать 'FirstName' для читаемости",
      "auto_fixable": true
    }
  ],
  "checks": {
    "lint_passed": true,
    "vet_passed": true,
    "style_guide_violations": 0,
    "complexity_score": 4.2,
    "max_allowed_complexity": 10
  },
  "decision_rule": "verdict = critical == 0 ? 'approved' : 'rejected'"
}
```

```
}  
}
```

4.6. Tester → Orchestrator: результат тестирования

```
{  
  "type": "tests.completed",  
  "source": { "agent_id": "tester" },  
  "target": { "agent_id": "orchestrator" },  
  "payload": {  
    "task_id": "task-001-a",  
    "verdict": "passed",  
    "test_run": {  
      "total": 12,  
      "passed": 12,  
      "failed": 0,  
      "skipped": 0,  
      "duration_ms": 3400  
    },  
    "coverage": {  
      "line_percent": 92.3,  
      "branch_percent": 85.7,  
      "function_percent": 100.0,  
      "threshold_met": true  
    },  
    "generated_tests": [  
      {  
        "file": "internal/domain/user_test.go",  
        "test_name": "TestUser_Validate_EmptyEmail",  
        "type": "unit",  
        "result": "passed"  
      },  
      {  
        "file": "internal/domain/user_test.go",  
        "test_name": "TestUser_Validate_InvalidPhone",  
        "type": "unit",  
        "result": "passed"  
      }  
    ]  
  }  
}
```

4.7. Security → Orchestrator: результат проверки безопасности

```
{
  "type": "security.completed",
  "source": { "agent_id": "security" },
  "target": { "agent_id": "orchestrator" },
  "payload": {
    "task_id": "task-001-a",
    "verdict": "passed",
    "scans": {
      "sast": {
        "tool": "gosec",
        "findings": 0,
        "critical": 0,
        "high": 0
      },
      "secrets": {
        "tool": "trufflehog",
        "findings": 0
      },
      "dependencies": {
        "tool": "govulncheck",
        "vulnerable_deps": 0,
        "total_deps_scanned": 14
      }
    },
    "compliance": {
      "owasp_top10_check": true,
      "injection_safe": true,
      "auth_check": "n/a"
    }
  }
}
```

5. Поток сообщений — Event Bus



5.1. Архитектура шины

```
// Порт (интерфейс) — use case layer
type EventBus interface {
    Publish(ctx context.Context, event AgentMessage) error
    Subscribe(ctx context.Context, topic string, handler EventHandler) error
    Unsubscribe(topic string, handlerID string) error
}

type EventHandler func(ctx context.Context, msg AgentMessage) error

// Топики по конвенции
// task.*      — жизненный цикл задач
// review.*    — результаты ревью
// tests.*     — результаты тестов
// security.*  — результаты проверок безопасности
// system.*   — health checks, метрики
// artifact.*  — создание/обновление артефактов
```

5.2. Реализация (infrastructure layer)

Primary: Go channels с fan-out паттерном — zero latency, zero external deps.

Fallback: Redis Streams — для persistence, replay, и multi-instance deployment.

```
// Infrastructure adapter
type ChannelEventBus struct {
    subscribers map[string][]subscriberEntry
    mu           sync.RWMutex
    bufferSize  int // default: 1024
}

// Redis fallback для production
type RedisEventBus struct {
    client    *redis.Client
    group     string
    consumer  string
    maxLen    int64 // stream max length, default: 10000
}
```

6. Clean Architecture — структура проекта

ИСТОК — Clean Architecture (Go)

Entities
Interfaces
Adapters
(Task, Agent, Artifact,
original, problem, solution, FSM,
Message)

→ Dependencies point inward. Inner layers know nothing about outer layers.

6.1. Структура каталогов

```
istok/
├─ cmd/
│   └─ istok/
│       └─ main.go           # Entry point, DI
│
├─ internal/
│   ├─ domain/              # ENTITIES (innermost layer)
│   │   ├─ task.go          # Task, TaskState, TaskPriority
│   │   ├─ agent.go         # Agent, AgentType, AgentCapability
│   │   ├─ artifact.go      # Artifact, ArtifactType
│   │   ├─ message.go       # AgentMessage, EventType
│   │   └─ errors.go        # Domain errors
│   │
│   ├─ usecase/             # USE CASES (orchestration)
│   │   ├─ orchestrator.go  # Main orchestration FSM
│   │   ├─ planner.go       # Task decomposition logic
│   │   ├─ executor.go      # Agent execution coordination
│   │   ├─ verifier.go      # Review/test aggregation
│   │   └─ ports.go         # Interface definitions
│   │
│   └─ adapter/             # INTERFACE ADAPTERS
│       ├─ grpc/
│       │   ├─ server.go    # gRPC handlers
│       │   └─ proto/       # Protobuf definitions
│       ├─ eventbus/
│       │   ├─ channel_bus.go # Go channel implementation
│       │   └─ redis_bus.go  # Redis Streams implementation
│       ├─ http/
│       │   └─ api.go        # REST API (health, tasks, status)
│       └─ presenter/
│           └─ task_presenter.go # Output formatting
│
└─ infrastructure/         # INFRASTRUCTURE (outermost)
    ├─ llm/
    │   ├─ openai_client.go  # OpenAI adapter
    │   ├─ anthropic_client.go # Anthropic adapter
    │   └─ local_client.go   # Ollama/vLLM adapter
    ├─ storage/
    │   ├─ postgres_repo.go  # PostgreSQL task repository
    │   └─ s3_artifact_store.go # S3 artifact storage
    ├─ observability/
    │   └─ tracing.go        # OpenTelemetry traces
```

```

|   |   | └─ metrics.go           # Prometheus metrics
|   |   | └─ logging.go          # Structured logging (slog)
|   |   └─ config/
|   |       └─ config.go          # Environment-based config
|   |
|   └─ pkg/                       # Shared libraries
|       └─ fsm/                   # Generic FSM engine
|           └─ fsm.go
|       └─ retry/                 # Retry + circuit breaker
|           └─ retry.go
|           └─ circuit_breaker.go
|       └─ idempotency/
|           └─ idempotency.go     # Idempotency key checker
|
|   └─ configs/
|       └─ agents.yaml            # Agent definitions & capabilities
|       └─ workflows.yaml        # Workflow templates
|
└─ go.mod
└─ go.sum
└─ Makefile

```

6.2. Правило зависимостей

```

Infrastructure → Adapters → Use Cases → Domain
      ↓           ↓           ↓           ↓
(знает всех)  (знает UC+D) (знает D)  (не знает никого)

```

- **Domain** — чистые Go-структуры, zero imports из других слоёв
 - **Use Cases** — бизнес-логика, зависит только от интерфейсов (ports)
 - **Adapters** — реализации интерфейсов, конвертация форматов
 - **Infrastructure** — конкретные технологии (PostgreSQL, Redis, OpenAI)
-

7. Error Handling & Resilience

7.1. Классификация ошибок

```
type ErrorClass string

const (
    ErrTransient ErrorClass = "transient" // Retry: таймаут, rate limit, 5xx
    ErrPermanent ErrorClass = "permanent" // Не retry: невалидный input, 4xx
    ErrResource  ErrorClass = "resource"  // Retry с backoff: OOM, диск, квота LLM
    ErrLogic     ErrorClass = "logic"     // Не retry: баг в агенте, assertion failed
)
```

7.2. Стратегия обработки

Класс ошибки	Retry?	Действие
transient	Да, exponential backoff	Повторить через 2s → 4s → 8s
permanent	Нет	Перевести задачу в FAILED, уведомить
resource	Да, с увеличенным таймаутом	Переключить на менее ресурсоёмкую модель
logic	Нет	Логировать stack trace, FAILED

7.3. Каскадная отказоустойчивость

```
Agent Failure
|
|→ Retry Policy (max 3)
|   |
|   |→ Success → продолжить
|   |→ Fail → Circuit Breaker check
|           |
|           |→ CLOSED → retry
|           |→ OPEN → fallback agent или BLOCKED
|           |→ HALF_OPEN → probe с лёгкой задачей
|
|→ Dead Letter Queue (DLQ) → ручной разбор
```

8. Observability

Каждое сообщение несёт `trace_id` и `correlation_id` , что позволяет:

Компонент	Инструмент	Что отслеживается
Tracing	OpenTelemetry + Jaeger	Полный путь задачи через всех агентов
Metrics	Prometheus + Grafana	Latency per agent, error rate, queue depth
Logging	slog (structured)	JSON-логи с <code>trace_id</code> в каждой строке
Alerting	Alertmanager	Circuit breaker trips, DLQ overflow, SLA breach

9. Конфигурация агентов

```
# configs/agents.yaml
agents:
  - id: coder
    type: executor
    model: gpt-4o
    fallback_model: claude-3.5-sonnet
    max_concurrent_tasks: 5
    timeout: 120s
    retry:
      max_attempts: 3
      base_delay: 2s
      backoff_factor: 2.0
    circuit_breaker:
      failure_threshold: 5
      reset_timeout: 60s
    capabilities:
      - code_generation
      - code_refactoring
      - test_generation
    languages:
      - go
      - typescript
      - sql
    tools:
      - go_vet
      - gofmt
      - ast_parser

  - id: designer
    type: executor
    model: dall-e-3
    fallback_model: stable-diffusion-xl
    max_concurrent_tasks: 3
    timeout: 180s
    capabilities:
      - ui_mockup
      - icon_generation
      - svg_creation
      - design_token_extraction
    output_formats:
      - png
```

- svg
- json

10. Типы событий (полный реестр)

Событие	Описание	Publisher	Subscriber(s)
<code>task.created</code>	Новая задача от пользователя	API Gateway	Orchestrator
<code>task.accepted</code>	Orchestrator принял задачу	Orchestrator	Planner
<code>task.decomposed</code>	Planner разбил на подзадачи	Planner	Orchestrator
<code>task.assigned</code>	Подзадача назначена агенту	Orchestrator	Coder/Designer/Writer
<code>task.completed</code>	Агент завершил работу	Coder/Designer/Writer	Orchestrator
<code>task.blocked</code>	Зависимость не разрешена	Orchestrator	— (internal)
<code>task.resubmitted</code>	Повторная отправка после ревизии	Coder/Designer	Orchestrator
<code>review.started</code>	Ревью начато	Reviewer	—
<code>review.completed</code>	Ревью завершено	Reviewer	Orchestrator
<code>tests.started</code>	Тесты запущены	Tester	—
<code>tests.completed</code>	Тесты завершены	Tester	Orchestrator
<code>security.completed</code>	Проверка безопасности завершена	Security	Orchestrator
<code>system.health</code>	Heartbeat агента	Все агенты	Orchestrator
<code>system.circuit_open</code>	Circuit breaker сработал	Orchestrator	Alerting
<code>artifact.created</code>	Новый артефакт	Любой агент	Storage
<code>task.cancelled</code>	Задача отменена	Orchestrator	Все
<code>task.failed</code>	Задача провалена	Orchestrator	Alerting

Методология

Источники: Azure Architecture Center (AI Agent Orchestration Patterns), GitHub Engineering Blog (Multi-Agent Workflows), паттерны из AutoGen/CrewAI (адаптированы для Go). Архитектурные решения основаны на принципах Clean Architecture (Robert C. Martin) и Hexagonal Architecture (Alistair Cockburn).

Ограничения: Это архитектурный blueprint, не production-ready код. Конкретная реализация LLM-клиентов, prompt engineering для каждого агента и интеграционные тесты требуют отдельного проектирования.

References

- ¹ Microsoft Azure Architecture Center — AI Agent Orchestration Patterns. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>
- ² GitHub Engineering Blog — Multi-agent workflows often fail. Here's how to engineer ones that don't. <https://github.blog/ai-and-ml/generative-ai/multi-agent-workflows-often-fail-heres-how-to-engineer-ones-that-dont/>
- ³ Robert C. Martin — Clean Architecture: A Craftsman's Guide to Software Structure and Design (2017)
- ⁴ O'Reilly — Designing Effective Multi-Agent Architectures (2025). <https://www.oreilly.com/radar/designing-effective-multi-agent-architectures/>
- ⁵ ArXiv — The Orchestration of Multi-Agent Systems: Architectures (2026). <https://arxiv.org/html/2601.13671v1>